

Filesystem (ex Carpeta Postán)

Agustín Merino, Esteban Ruiz

Mayo 2026

1. Concepto

Un sistema de archivos (File System) sirve para gestionar información persistente. Debe permitir:

- Guardar información
- Consultarla
- Actualizarla
- Eliminarla

El sistema de archivos es, en realidad, una abstracción provista por el SO que le da semántica al disco. El disco tiene todos sus datos como bytes puros. Un sistema de archivos le permite al SO establecer un orden en dicho disco. Sin él, el sistema no sabría cómo interpretar los datos en el medio de acceso que se encuentran.

Típicamente se usan medios magnéticos. Pueden ser cintas, discos magnéticos, de estado sólido, etc.

La información en dichos discos se almacena a través de bloques, los cuales son conjuntos de N bytes, típicamente unos 512 bytes. Todas las transferencias (lecturas, escrituras) se realizan de a bloques, no de a bytes individuales.

Todo sistema de archivos contará con dos cosas centrales:

- La información que tiene que guardar (data).
- La información extra para la gestión (metadata). Esta generalmente consta de fecha de creación, tamaño, nombre, etc..

A continuación se presentan algunos sistemas de archivos.

Aclaración: Se utilizará la palabra *disco* para referirse al medio físico de almacenamiento. Este puede no ser necesariamente un disco en sí, sino una cinta o un medio de estado sólido.

2. Tipos de FS

2.1. TAR (UNIX)

Tiene la siguiente estructura

Tiene como referencia las unidades de cintas. A medida que voy guardando archivos en el disco, debo guardar su cabecera (*header*) la cual contiene la metadata, y posteriormente, sus datos. La cabecera ocupa un bloque.

El relleno * se usa en caso de que el archivo no llegue a ocupar bloques completos. Los bloques no se comparten entre archivos distintos. Como se puede notar, esto ocasiona que siempre haya fragmentación interna.



Figura 1: Estructura del sistema de archivos TAR

¿Qué sucede si quiero modificar un archivo? Si hay espacio para mi modificación en el bloque asignado a los datos del archivo, simplemente los escribo. El problema viene cuando no hay espacio suficiente y debo agregar más bloques. A menos que el archivo se encuentre al final del disco, debería marcar el archivo como borrado y reescribirlo al final.

- Ventajas
 - Simple.
- Desventajas
 - Acceso secuencial.
 - Actualización compleja.
 - Si los bloques del medio de almacenamiento son mayores a 512 Bytes, los archivos estarán en un sub-bloque.
 - Para saber si un archivo está presente en el disco, debo, potencialmente, recorrerla en su totalidad, cabecera por cabecera.

Es útil para ocasiones donde las actualizaciones no son necesarias.

2.2. Commodore 64 (concentrar la metadata)

Debido a la última desventaja del sistema TAR, se propuso como idea concentrar toda la metadata, o sea, los *headers*, al comienzo del disco.

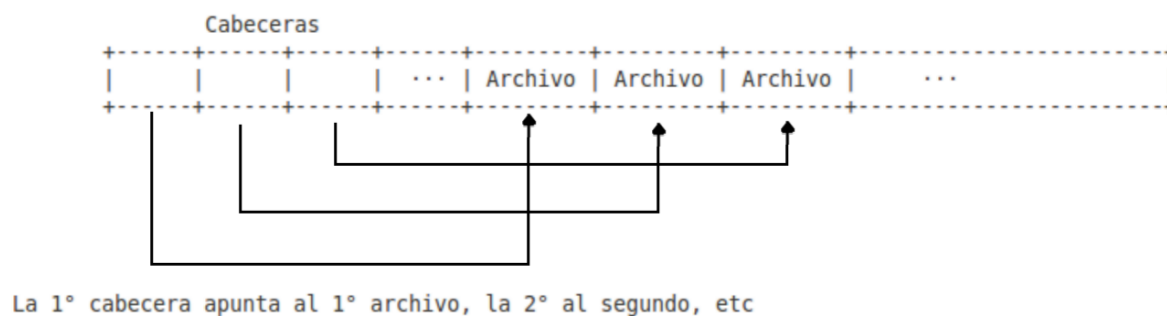


Figura 2: Commodore 64 File System.

De esta manera, para verificar si un archivo está presente (o cualquier metadato de algún archivo) se obtiene leyendo los primeros bloques del disco.

2.3.2. Más detalles del QuestarFS

Las direntries contenían la información que en los casos anteriores llamabamos cabeceras.

El nombre podía ser de 7 caracteres máximo. Los 3 que se suman son la extensión (el punto no se guardaba).

La clave de acceso se guardaba en ASCII y daba una forma de obtener un acceso controlado al mismo.

El archivo se componía de fragmentos. Cada fragmento en el header constaba del sector de inicio y un número indicando cuántos fragmentos consecutivos a este existían. Es decir, si el 1er sector era el 200 y se tenían 6 sectores consecutivos, ese fragmento era el conjunto de sectores $\langle 200, 201, 202, 203, 204, 205 \rangle$. Luego, el 2do fragmento no iniciaba en el 206 (fragmentación).

Cuando se agrandaba un archivo (más allá de su último sector), se veía si se podía agregar en el último fragmento. Para que pudiera hacerse, debía haber sectores libres a continuación del último sector del fragmento. En caso contrario se creaba un nuevo fragmento.

Una desventaja, al igual que en los casos anteriores, es que existe fragmentación interna en el caso que un archivo no use un sector completo.

Problema: Fragmentación acotada. Una vez que se superan los n fragmentos, ya no puede seguir fragmentándose más el archivo.

NOTA: Un subdirectorio es simplemente un archivo cuyos sus fragmentos son DirEntries.

2.4. MSDOS FS

A principios de los 80, Microsoft comienza a utilizar este File System.

Los datos de este File System, no se organizan en sectores, sino más bien en **clusters**, es decir **grupos de sectores**: cada cluster contiene 4 sectores, y cada sector 512 Bytes. Por lo tanto, **cada cluster es de 2048 Bytes (2KB)**.

El funcionamiento se basa en unificar dos aspectos a manejar:

- Gestión del espacio libre.
- Permitir y gestionar la fragmentación arbitraria. Es decir, no tener un número de fragmentos acotados, como se tenía en QuestarFS.

Para lograr esto, se utiliza como estructura principal una estructura denominada FAT (*File Allocation Table*, tabla de ubicación de archivos).

Dicha estructura tiene esta forma:

Los clusters representados en la FAT son enteros de N bytes, donde N depende del tamaño de la FAT (FAT-16, FAT-32, etc..) y pueden tener los siguientes valores (ejemplo con 24 bits):

- **0**: El cluster está libre.
- **FFFFFFE (-2)**: Cluster dañado (seteado durante el formateo).

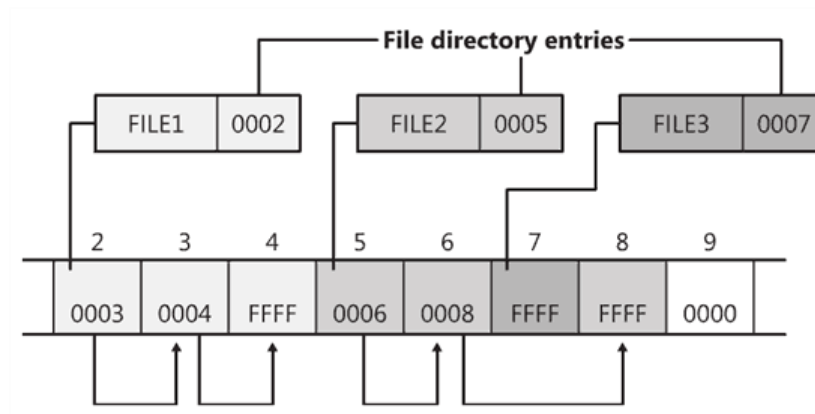


Figura 4: File Allocation Table

- **FFFFFF** (-1): Último cluster de un archivo.
- **Otro**: Cluster en uso por un archivo. El número asignado indica el cluster donde continúa el archivo.

La FAT permite representar la información de los clusters de un archivo como una lista enlazada.

Las escrituras en la FAT deben ser **síncronas** para evitar un apartado de problemas relacionados con la inconsistencia del disco (ver libro “Fundamentos de Sistemas Operativos”).

La estructura general del MSDOS File System, que utilizaba la FAT era:

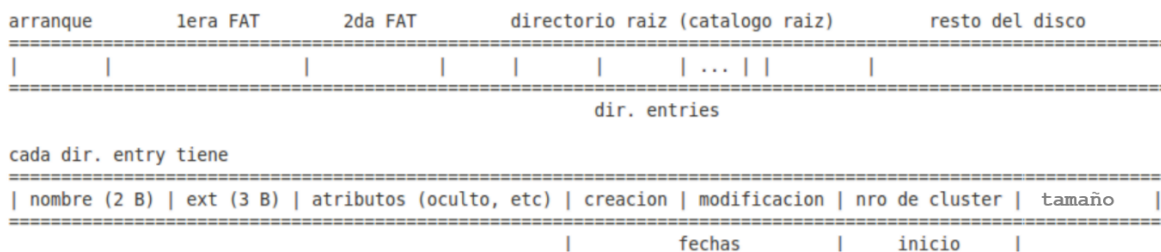


Figura 5: MSDOS File System

En el cluster de arranque se guardan los datos necesarios para el booteo del sistema (firmware UEFI/BIOS).

Se guardan 2 copias de la FAT para seguridad, si se llega a dañar el bloque donde se encuentra la primera FAT, se perdería toda la estructura de los archivos.

Los metadatos se guardan en un directorio raíz (y sub-directorios), sólo que cada header no contiene información de los fragmentos: en su lugar posee la dirección del primer cluster que contiene los datos del archivo. A partir de este cluster, el SO puede ir resolviendo la ubicación de los siguientes haciendo uso de la FAT.

- Ventajas

- Es un buen FS para discos chicos.
- Desventajas
- La FAT ocupa mucho espacio y hay que mantenerla siempre en memoria principal. Cada entrada de la FAT es un entero (ej. 4 bytes) por cada cluster del sistema. Si el disco es de 20GB con un tamaño de sector de 2 KB, la tabla necesitará 10 millones de entradas, una para cada uno de los 10 millones de clusters. Como cada entrada almacena un entero de 4 bytes, la FAT terminará pesando unos 40 MB. Esto será, tener ocupados 80 MB de memoria ram todo el tiempo. Puede ponerse en memoria paginable (podría escribirse en SWAP), pero seguiría ocupando una gran cantidad de memoria virtual y espacio en disco, así como generaría tráfico de paginación extra.
 - La política de actualización síncrona de la FAT ralentizaba la escritura.

2.5. UNIX FS

Se comenzó a desarrollar a principios de los 70, antes de FAT. Es la base de la mayor parte de los sistemas de archivos basados en los derivados de Unix, Linux, etc...

Su concepto principal es el **i-nodo** (*information node*). La estructura del disco es la siguiente:

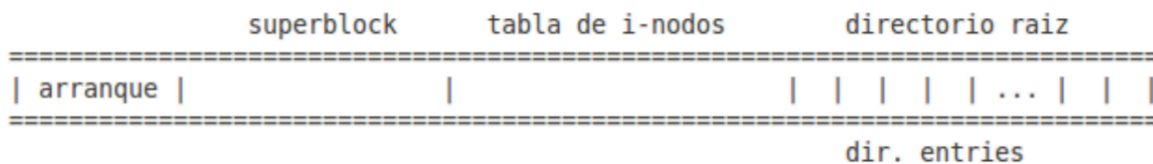


Figura 6: UNIX File System

Al principio, al igual que MSDOS, tendremos el arranque, espacio del disco reservado para los programas cargadores del SO (bootloaders). ¹.

Luego, sigue el **superblock**, el cual contiene información sobre todos los parámetros del disco, como está formateado, cantidad de i-nodos, última fecha de montaje, número mágico que identifica el tipo de sistema de archivos, etc... (información administrativa).

Particularmente, el superblock tiene:

- El bitmap.

¹UEFI, la nueva versión de BIOS, es una especificación que define una interfaz entre el sistema operativo y el firmware de la plataforma. El firmware es el programa embebido en la motherboard encargado de leer y ejecutar los primeros 512Bytes del disco de arranque, que es usualmente donde se encuentra el bootloader del SO. Además, se encarga de funciones como: inicializar hardware, mostrar el setup, buscar el dispositivo de arranque, etc..

- Lista enlazada de i-nodos libres.

La tabla de i-nodos posee información de todos los i-nodos del sistema, es decir, de todos los archivos, ya que necesito un i-nodo por cada archivo.

Finalmente, se encuentra el root (directorio raíz), el cual contiene todas las direntries (del directorio raíz). Sin embargo, en este File System, las direntries son mucho más sencillas debido a que la mayor parte de los metadatos se guarda ahora en los i-nodos.

Se define ahora el concepto de **i-nodo**.

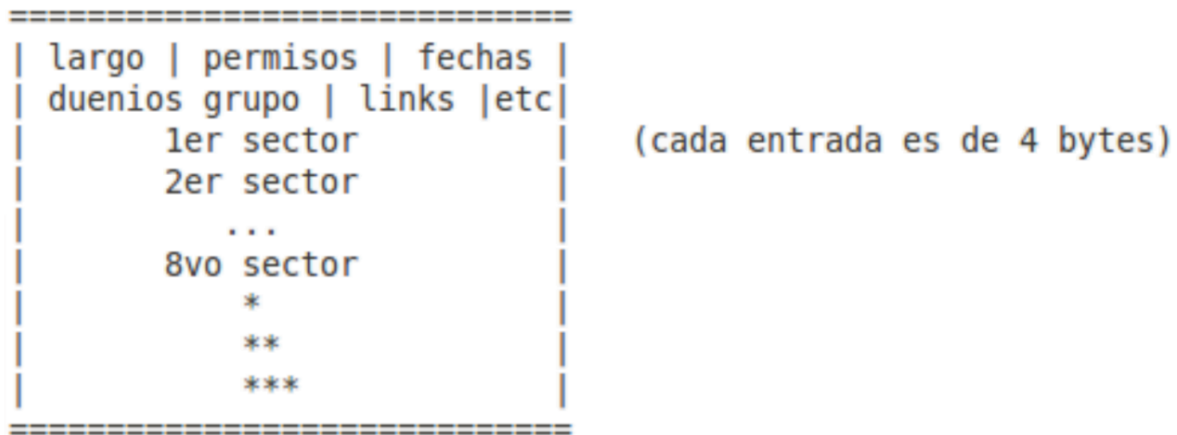


Figura 7: I-Nodo

Un **i-nodo** es una estructura que generalmente posee un tamaño fijo (un bloque - 512 Bytes) y posee toda la metadata de un archivo.

Un campo interesante es el de los links. Vemos que ahora las cabeceras no mantienen más el nombre del archivo. Ahora un archivo pueden tener distintos nombres (incluso idéntico nombre en dos directorios diferentes). Lo que importa es que el archivo creado haga referencia al i-nodo correspondiente en el disco. El campo links dice cuántas referencias actuales tiene el archivo. Si dicho campo llega a 0, significa que el archivo no será más direccionable y puedo liberarlo (se borró el archivo). Este campo trata sobre los **hard links**, no los **soft links**.

El i-nodo contiene una lista de sectores direccionables directamente.

* es un puntero que apunta a una tabla de 128 entradas de 4 bytes (un entero) que indica dónde se encuentran los siguientes sectores (en este caso el 9, 10, 11,...).

** Se conoce como 2 indirecciones. Básicamente apunta a una tabla como * pero, en vez que sus entradas apunten a sectores, apuntan a tablas del tipo *. Es decir, tiene 128^2 sectores. Esta hay que implementar en NachOS.

*** Se conoce como 3 indirecciones y sigue la misma idea.

En este diagrama de un i-nodo, puede verse más detalladamente:

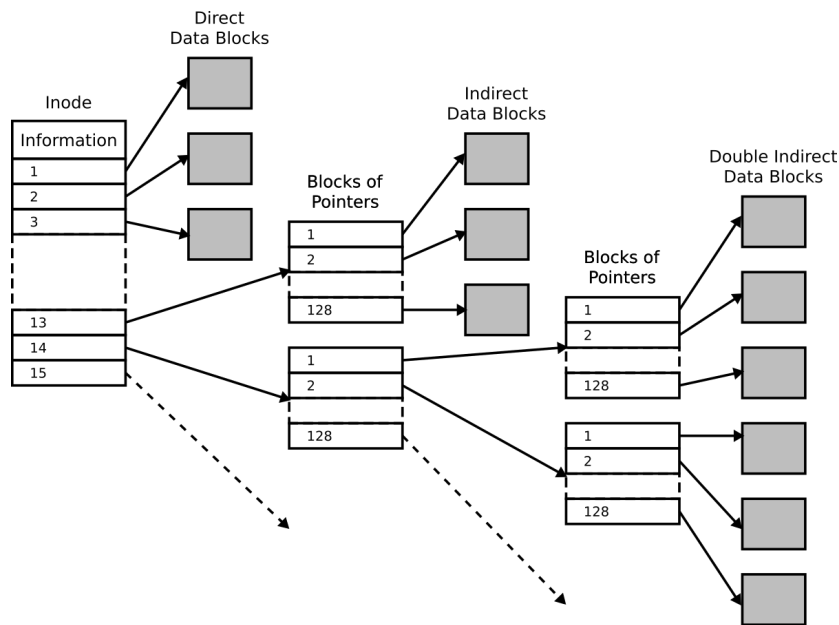


Figura 8: Niveles de indirección del i-nodo en detalle.

Por otro lado, las DirEntries, son de la forma:

```

=====
| nombre (14 bytes) | i-node |
=====

```

Figura 9: Direntry en UnixFS

Vemos que guardan el nombre del archivo y el número de su i-nodo, mediante el cual se puede buscar en la tabla de i-nodos. Una vez encontrado el i-nodo, ya tengo toda la información necesaria para manipular el archivo. En UNIX no existe el concepto de “extensión.”^{en} el nombre de archivo, algunos programa usan el sufijo -los caracteres después del último “. para discriminar archivos.

Vemos que incluso, un archivo puede tener distintos nombres, al contrario que FAT. Pueden haber muchas entradas de directorios distintas que apunten al mismo i-nodo. Por lo tanto, si tengo dos archivos con dos nombres distintos (dos DirEntries distintas) que apuntan al mismo i-nodo, cada vez que modifique algo en uno de ellos, se reflejará en el otro (debido a que se trata del mismo i-nodo).

Vemos entonces que la cuenta de links en el i-nodo representa el número de cabeceras apuntando a dicho i-nodo.

Borrar un archivo ahora es borrar una direntry. Cada vez que borro una direntry, bajo la cuenta de links del i-nodo. Cuando llega a 0, se puede reclamar todo el espacio del archivo (sectores ocupados) como libres en el bitmap.

Recordar que el i-nodo ocupa un sector, por lo que también tendrá un lugar en el bitmap.

3. Mejorando el FS

El sistema de archivos introducido por **BSD** ², basado en Unix FS introdujo montones de mejoras, principalmente de rendimiento, las cuales hicieron que durante mucho tiempo fuera considerado un sistema de archivos de referencia.

3.0.1. Hacer crecer el sector lógico

BSD introdujo el concepto de **sectores lógicos** distintos de los sectores físicos. Vemos que FAT agrupa lógicamente sectores en **clusters**. Aquí se hace algo muy parecido llamándolos **sectores lógicos**. Un sector lógico en BSD era de 4096 Bytes (4KB) en lugar de 512 Bytes que tiene un sector físico. Aunque los programas lean de a un sector (512 Bytes), el SO pedía un sector lógico (4KB - 8 sectores). De esta forma, cuando el programa pedía el próximo sector, este ya se encontraba en memoria. Esto aportó una mejora del 20 % en la velocidad.

3.0.2. Algoritmo del ascensor

Para explicar esta mejora, primero veamos la forma del medio de almacenamiento físico basado en discos magnéticos, los cuales, por el momento, siguen siendo ampliamente utilizados.

Podemos ver dicho dispositivo en las siguientes figuras:

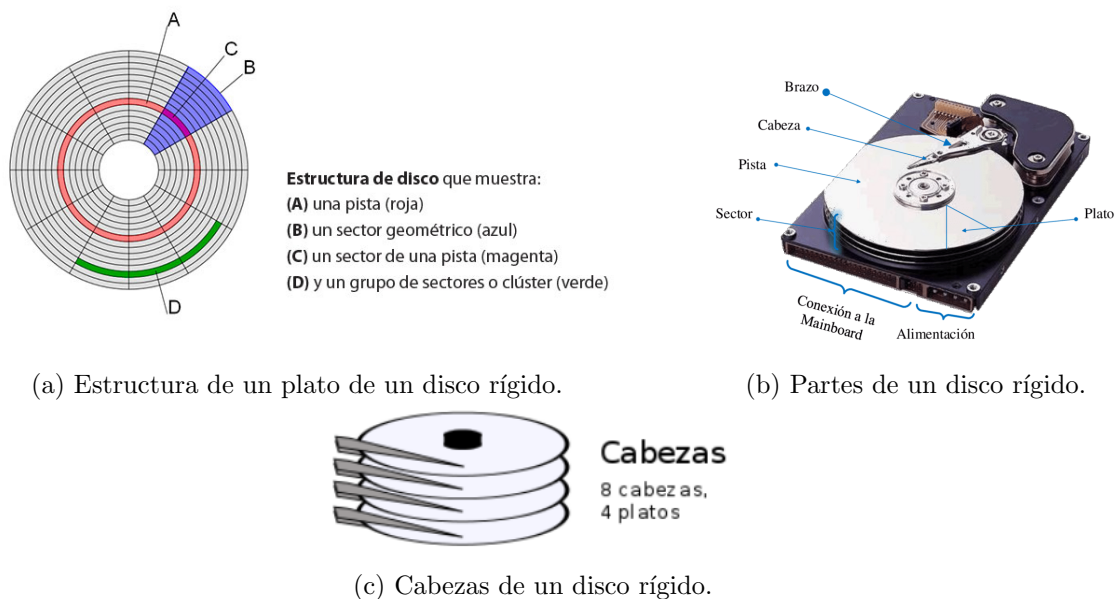


Figura 10: Formato de un disco rígido.

Los platos contienen todos los bytes del disco. Giran todos juntos, a la misma velocidad,

²BSD empezó como un conjunto de mejoras al Unix original de AT&T, pero con el tiempo se convirtió en un sistema completo por derecho propio. Muchas ideas y componentes que nacieron en BSD terminaron siendo adoptadas por otros sistemas Unix (y también por sistemas no-Unix como Windows y macOS).

y las cabezas (especie de púas duales, una para la parte superior de cada plato y otra para la inferior) van leyendo todas al mismo tiempo, en el mismo lugar en los distintos discos.

Manteniendo las cabezas fijas, se leen todos los datos de una misma **pista**. El conjunto de pistas de todos los platos forman un **cilindro** si dejamos fijo el número de pista. Dentro de una **pista**, se puede grabar/leer información dentro de *sectores*. Es decir, una **pista** está formada por un **conjunto de sectores**.

A su vez, las cabezas poseen un pequeño motor que permite modificar su ángulo, permitiendo desplazarse entre las pistas.

Por lo tanto, para referenciar a los bloques de un disco rígido se usa un método **CHS** (*Cylinder, Head, Sector*).

Notar que los sectores son todos del mismo tamaño y el diámetro de las pistas va variando. Esto significa que las pistas más cercanas al centro tendrán los datos más comprimidos dentro de los sectores, mientras que en las más alejadas del centro, los sectores presentarán sus datos más dispersos. Actualmente, se han agregado “métodos” que no requieren dispersión en los datos de las pistas con mayor diámetro aprovechando así mucho más el espacio.

En resumen, leer todos los datos que están dentro de una pista, es decir, todos los sectores, es muy rápido (si un disco gira a 7200 rpm, en 1/7200 de minuto - algo así de 8.33 milisegundos - puedo leer una pista completa). Sin embargo, mover el cabezal (se mueve con un motor paso a paso) es muy lento en comparación.

Cabe aclarar que esta forma de organizar los datos en el disco ya quedó **obsoleta**. Existen nuevos métodos y dispositivos mucho más rápidos.

¿Qué propuso BSD en base a esto?

Comencemos con un ejemplo, supongamos que tenemos que acceder (independientemente de lectura o escritura) a las siguientes pistas (se debería usar mecanismo CHS pero nos interesa evaluar el rendimiento):

$$\langle 1, 10, 5, 2, 9, 3 \rangle$$

Creemos la secuencia con la cantidad de pistas que hay que desplazarse para pasar de la pista i a la pista $i + 1$, asumiendo que comenzamos en la pista 1:

$$\langle 9, 5, 3, 7, 6, 9 \rangle$$

Es decir, de la 1 a la 10, hay 9 pistas de distancia, de la 10 a la 5, hay 5, etc..

Sumando esta secuencia, vemos que la cantidad de pistas que tenemos que saltar para hacer todos los pedidos, es:

$$9 + 5 + 3 + 7 + 6 + 9 = 39$$

Esto quiere decir, que en el orden de pistas propuesto, deberemos recorrer 39 pistas para poder realizar los pedidos.

La mejora de BSD fue, **ir escribiendo en caché todos los accesos necesarios (guardarlos en memoria) y ordenarlos para evitar largos recorridos.**

Es decir, los accesos ahora serán:

$$\langle 1, 2, 3, 5, 9, 10, 12 \rangle$$

y la secuencia con la cantidad de pistas a recorrer ahora es:

$$\langle 1, 1, 2, 4, 1, 2 \rangle$$

Es decir, un total de:

$$1 + 1 + 2 + 4 + 1 + 2 = 11$$

Terminaremos recorriendo en total 11 pistas.

A este algoritmo se le conoce como **algoritmo del ascensor** debido a su modo de funcionamiento al ir recorriendo los pisos en orden (si aprieto los pisos 1,10,3,7 comenzando del 0, cualquier ascensor mínimamente funcional recorrerá en orden: 1,3,7,10).

Sin embargo, esto no siempre es óptimo. **El algoritmo del ascensor nos garantiza su optimalidad en tanto los pedidos de lectura/escritura sean independientes.**

Por ejemplo, imaginemos que estamos borrando un archivo utilizando UnixFS.

Los pasos a seguir son:

1. Deberé buscar el archivo. Para eso deberé recorrer el directorio correspondiente buscando el nombre del archivo para así obtener su i-nodo.
2. Deberé modificar el directorio para marcar la entrada de directorio correspondiente al archivo borrado como libre.
3. Finalmente, debo ir al i-nodo, y si la cuenta de links pasa a 0, debo liberar el i-nodo (marcarlo como libre en la tabla de i-nodos) y marcar como libres en el bitmap todos los sectores presentes en dicho i-nodo.

Pero, ¿qué sucede en caso de falla entre el paso 2 y 3? **Existirá un i-nodo que no está referenciado desde ningún directorio y, por lo tanto, jamás podré reclamar los sectores correspondientes al archivo y habré perdido espacio.**

Esto, sin embargo, no es demasiado problema ya que existen métodos que permiten verificar el disco y buscar i-nodos **huérfanos** (i-nodos sin nombre que los referencie). A estos

i-nodos los puedo intentar **recuperar**, lo cual, en caso que el sistema no pueda reconstruir su estructura general los guardará en un directorio especial llamado **lost+found** (se explica mejor más adelante), o simplemente el sistema puede reclamar el espacio asumiendo que pertenece a un archivo eliminado.

Pero ¿Qué sucede si hubiera hecho las cosas al revés y el sistema falla? Es decir, invirtiendo 2 y 3 y el sistema fallando al terminar la nueva 2. En este caso, habré marcado el i-nodo como libre pero no liberé su entrada de directorio. Es decir, en mi directorio todavía existe un archivo que apunta al i-nodo libre. Si se crea un nuevo archivo, y se le da dicho i-nodo, tendré dos archivos no relacionados pero apuntando al mismo i-nodo y, por lo tanto, todos los cambios que haga sobre uno se reflejarán en el otro.

En este caso, el sistema de archivos queda mucho más corrupto.

Entonces, en operaciones como creación o eliminación de archivos, es importante que los metadatos se escriban en un determinado orden.

Otra cosa que se implementó fue el llamado **interleaving**. A grandes rasgos la idea es que: los discos giran tan rápido que si los sectores están contiguos puedo pasarme de alguno que necesitaba leer y tener que dar toda la vuelta. Es por esto que puedo dejar un espacio entre sectores contiguos para equiparar los tiempos de lectura/escritura y revoluciones del plato.

3.0.3. Distribución del superblock, y demás estructuras

Muy probablemente, a medida que use el disco, estando en cualquier parte del mismo, necesite información del superblock o de la tabla de i-nodos. Recordemos que estos se encuentran al principio del disco, en los primeros sectores. Es por esto que BFS optó por distribuir estas estructuras a lo largo de todo el disco. Sin embargo, no se tratan de copias exactas de dichas estructuras, sino que existe una noción de *localidad*. Es decir, se tiene el superblock, bitmap y tabla de i-nodos específicos a cada grupo de cilindros cerca de estos. De esta forma, se obtiene una ganancia en velocidad ya que la información del disco siempre está relativamente cerca del lugar de trabajo.

Esta distribución no se hace a múltiplos exactos, de forma que si se rompe alguna pista/-cabeza no se encuentren todos los datos alineados y se arruinen todas las copias.

Notar que hacer esto también provee mayor seguridad. Tener distintas copias de estas estructuras primordiales para nuestro FS previene que la pérdida de alguna de estas sea algo irreversible.

Estos cambios dieron una mejora de 60 % en velocidad.

Al sistema de archivos de BSD se le llamó **FFS** (*Fast File System*).

4. Más mejoras

4.0.1. Eliminando secuencias dependientes

Vimos que el algoritmo del ascensor se rompía en caso que las operaciones sean dependientes entre sí. Podemos intentar quitar dichas dependencias para aprovechar al completo el algoritmo del ascensor.

Esto fue lo que introdujeron los FS *ext* y *ext2*. Logró ignorar las secuencias dependientes estableciendo mecanismos o programas que revisan el disco completo buscando estructuras inconsistentes en el FS y solucionar las inconsistencias. Por ejemplo, el programa **fsck** (*file system check*) de los *ext* es mucho más complicado.

4.0.2. Cambiando la estructura del directorio

Tomemos la apertura de un archivo. El kernel debe:

- Buscar en el directorio correspondiente (un directorio es un array de direntries $\rightarrow O(n)$).
- Sacar su i-nodo de información.
- Checkear los accesos.
- Construir el descriptor en el System Segment.

Podemos acelerar el primer paso cambiando el array de direntries por otra estructura. Por ejemplo, un árbol. Si está balanceado el costo de búsqueda ahora será $O(\log n)$. Algunos casos de estos árboles son *Red-Black Tree*, *AVL* y *BTree*.

Los *BTree* (*Balanced Trees*) fueron elaborados en 1971 bajo la premisa de: “En vez de hacer crecer un árbol desde la raíz, lo hacemos crecer desde las hojas”.

Podemos utilizar dos *BTree*, uno para los directorios y otro para sectores libres (ordenados por tamaño y ubicación).

El pionero en usar *BTrees* fue IRIX (el Unix de SGI). Tener dos *BTree* para los sectores libres es complicado (deben ser consistentes entre sí) y eso lo llevó a tener que incluir **journaling** (*bitácora*).

La idea es que antes de realizar algún cambio, lo escribo en la bitácora. La misma contiene todas las acciones que voy a hacer, pero todavía no hice y se guarda en un sector del disco. Una vez que realizo los cambios, marco en la bitácora que todo anduvo bien. Por lo tanto, ante una eventual falla del sistema, cuando se reanuda su funcionamiento, se mira la bitácora para ver qué cambios no marqué como hechos y poder hacer algo al respecto (realizarlos, por ejemplo).

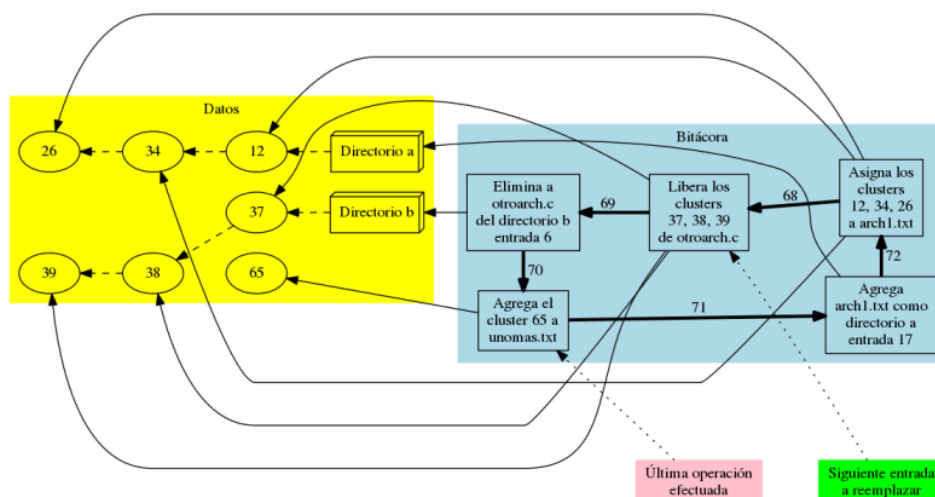


Figura 11: FS con bitácora.

El journaling, al ir escribiendo dos veces cada cambio, baja la performance del FS, a costa de aumentar la seguridad.

Es por eso que muchos sistemas de archivos usan journaling únicamente para los **metadatos**. En lugar de registrar los cambios que hace cada programa en particular sobre sus archivos, registro únicamente los cambios en los metadatos de los archivos que son los más importantes al momento de mantener el FS en un estado consistente.

FS con journaling: IRIX(xfs), IBM(jfs), ext3 (ext2 de Linux modificado para journaling), ext4, ResierFS (FS basado en BTree que implementaba cosas como compartir una página con varios archivos), etc..

¿Podemos seguir mejorando esto?

Vimos entonces que cambiando la estructura del directorio de un array a un árbol, mejorábamos la performance. Podemos mejorarlo aún más utilizando una **tabla hash**.

- Array $\rightarrow O(n)$
- BTree $\rightarrow O(\log n)$
- Hash $\rightarrow O(1)$

Ahora podemos tener una tabla hash de todos los archivos del directorio y buscarlos en $O(1)$. Sin embargo, no es tan práctico ya que cuando actualizo el directorio, debería actualizar la tabla hash.

Por otro lado, la **shell** de Linux usa este método para mantener un listado de los ejecutables. Cuando se pone un comando en la línea de comandos, no hay que buscarlo por todo el path, sino que se busca en la hash y puede hacerse en $O(1)$.

Por lo tanto, tener una tabla hash sirve para el caso que las ubicaciones de mis archivos no varíen demasiado (de hacerlo, se debería cambiar la hash recurrentemente).

Este esquema fue usado por BeFS de BeOS usado por BeBox.

5. Mejoras experimentales

Lo que buscamos entonces es:

- Máxima velocidad
- Máxima seguridad

El problema es que estos dos puntos son antagónicos. Generalmente, lo que ganamos de uno, perdemos del otro. En el caso del journaling por ejemplo, ganamos en seguridad escribiendo los cambios previamente en la bitácora, pero perdemos velocidad al tener que escribir todo dos veces.

Podríamos, por otro lado, hacer que nuestro FS sea, al mismo tiempo, la bitácora. Este caso se llama *log-structured File System* (LogFS) y fue presentado por primera vez en una tesis de doctorado de Margo Seltzer en Berkeley.

La idea de esto es, dividir el disco en segmentos e ir escribiendo los datos unos detrás del otro. Vemos que los sistemas de archivos anteriores, ubicaban los datos de tal forma que estén cerca de las diversas instancias de estructuras importantes tales como el superblock o el bitmap para así acelerar los tiempos de escritura de los mismos en el disco. LogFS rompe con esto debido a su hipótesis que dice que a medida que aumentan los tamaños de las memorias principales (RAM) en las computadoras modernas, las operaciones I/O serán únicamente pesadas en la escritura, ya que la lectura podrá tenerse casi siempre en caché (en memoria principal). Por lo tanto, LogFS plantea que el disco entero sea una especie de búffer circular que, a su vez, auspicie de log (bitácora) e ir escribiendo los cambios siempre en la cabeza de este búffer.

Pero, ¿Qué sucede con los i-nodos, bitmap, tabla de i-nodos, etc..? La idea es que, al terminar una escritura, pegado a donde se terminan los datos que escribí, se guardan todas estas estructuras ahora actualizadas. Es decir, cada operación deberá escribir nuevamente dichas estructuras para actualizarlas y agregarlas al final. Esto se conoce como ***copy-on-write***, en vez de modificar un bloque, lo copio en otro lado.

Al ser ahora todo secuencial, se mejoran enormemente las lecturas. ¿Pero y las escrituras, quedan penalizadas? Vemos que ahora, los datos, terminan quedando relativamente cerca de los metadatos, lo cual hace que las escrituras no se penalicen demasiado.

Sin embargo, acá vemos un problema... Jamás borramos, va agregando datos. Es decir, tendremos información vieja de la tabla de i-nodos, bitmap, superblock, etc... (o sea basura). Este es un inconveniente de este sistema de archivos. Periódicamente requiere el uso de un *garbage collector* que recorra el disco y pueda ir eliminando dicha información ahora inútil en los segmentos utilizados (que no sea el último, que sería el actual).

5.1. Algunos sistemas archivos actuales

- **ext4**: Es una mejora a *ext3* pero ya se dice estar quedando obsoleto (son parches sobre parches ya que el *ext3* es una mejora sobre *ext2* y *ext2* es una mejora sobre *extFS*). Las principales mejoras con respecto a *ext3*, son:
 - Soporte de volúmenes de hasta 1024 PiB (discos más grandes).
 - Soporta *extents*.
 - Menor uso del CPU.
 - Mejoras en la velocidad de lectura y escritura.

Otra cosa que agregan los nuevos sistemas de archivos es el concepto de ***snapshots*** (se explica más adelante). La idea básicamente es tomar una “foto” del estado del sistema en un momento particular y guardar la información necesaria para que los bloques mostrados no se liberen y que los bloques nuevos se escriban en otro lado.

- **Btrfs** (*B-tree FS*): Este FS está llamado a ser el sucesor del sistema *ext4*. Está basado en árboles binarios y tiene soporte para cosas tales como *snapshots*, *copy-on-write*, además de presentar mejoras y optimizaciones para discos *SSD*.

Este FS ya no sería un parche sobre parche como vinieron siendo los *ext*, sino que un FS completamente nuevo.

6. Comando útiles de Linux con respecto al FS

Comando	Descripción
<code>stats</code>	No es un comando estándar. A veces es un alias o script personalizado para mostrar estadísticas del sistema.
<code>df -i</code>	Muestra el uso de inodos por sistema de archivos. Útil para saber si se agotaron los inodos antes que el espacio.
<code>mkfs.ext2</code>	Formatea una partición con el sistema de archivos EXT2 (sin journaling).
<code>mkfs.ext3</code>	Formatea una partición con EXT3, versión de EXT2 con journaling.
<code>mkfs.ext4</code>	Formatea con EXT4, sistema moderno con journaling, extents y mejor rendimiento.
<code>mkfs.btrfs</code>	Crea un sistema de archivos Btrfs, que permite snapshots, compresión y otras características avanzadas.
<code>dumpe2fs</code>	Muestra información detallada de un sistema de archivos EXT2/3/4.
<code>debugfs</code>	Herramienta para examinar y modificar sistemas de archivos EXT2/3/4 a bajo nivel.

Cuadro 1: Descripción de comandos relacionados con sistemas de archivos en Linux

7. Sistemas de archivos modernos

Los sistemas de archivos modernos incorporan mecanismos avanzados de integridad, tolerancia a fallos y optimización para dispositivos SSD/NVMe.

7.1. Copy-on-write (CoW)

El mecanismo *copy-on-write* evita sobrescribir bloques existentes. En su lugar, las modificaciones se escriben en nuevos bloques y luego se actualizan los punteros correspondientes. Esto mejora la tolerancia a fallos y facilita la implementación de *snapshots*.

7.2. Checksums e integridad

Sistemas modernos como ZFS y Btrfs utilizan *checksums* para detectar corrupción silenciosa de datos. Esta técnica permite verificar que los bloques leídos coincidan exactamente con los escritos originalmente.

7.3. Optimizaciones para SSD

Los SSD poseen características distintas a los discos magnéticos tradicionales:

- No poseen partes móviles.
- El acceso aleatorio es mucho más rápido.
- Las escrituras tienen desgaste físico.

Por este motivo, muchos sistemas de archivos modernos implementan:

- *TRIM*
- *Wear leveling*
- Escrituras secuenciales
- Recolección de basura (*garbage collection*)

8. Bibliografía

- Arpaci-Dusseau, R. y Arpaci-Dusseau, A. *Operating Systems: Three Easy Pieces (OS-TEP)*.
- Silberschatz, A.; Galvin, P.; Gagne, G. *Operating System Concepts*.
- Tanenbaum, A. *Modern Operating Systems*.
- McKusick, M. *The Design and Implementation of the FreeBSD Operating System*.